

Web Services And Applications (WSAA) Project

Gerry Callaghan

student number: G00472971

The [README.md](#) contains much of the description as to where all my files are.

In summary, all project files are in: <https://github.com/callagg2/wsaa/tree/main/project>. Because I deployed my API to <https://callagg2.pythonanywhere.com/>, I also have a copy of the necessary files <https://github.com/callagg2/deploytopythonanywhere2> where I can sync with pythonanywhere should I make any changes. My endpoints for my API are highlighted in the table below:

Action	Method	URL	Sample Params	Sample Return	Sample Curl
Get all	GET	/routes	none	[{...}, {...}, {...}]	curl http://callagg2.pythonanywhere.com/ routes
Find by id	GET	/routes/id	none	{ "id": "7", "destination": "xxx", "route_map": "xxx", "distance": "xxx", "elevation": xxx }	curl http://callagg2.pythonanywhere.com/ routes/7
Create	POST	/routes	{ "destination": "xxx", "route_map": "xxx", "distance": 100, "elevation": 800 }	{ "id": "8", "destination": "xxx", "route_map": "xxx", "distance": 100, "elevation": 800 }	curl -H "content-Type:application/json" -X Post -d "{ "destination": "xxx", "route_map": "xxx", "distance": 100, "elevation": 800 }" http://callagg2.pythonanywhere.com/ routes
Update	PUT	/routes/id	{ "distance": 110 }	{ "id": "7", "distance": 110 }	curl -H "content-Type:application/json" -X PUT -d "{ "distance": 110 }" http://callagg2.pythonanywhere.com/ routes/7
Delete	DELETE	/routes/id	none	{ "Done": true }	curl -X delete http://callagg2.pythonanywhere.com/ routes/8

My database “wsaa” on Pythonanywhere comprises one table called “routes_table” and is structured as follows:

ID	Destination	Route_map	Distance	Elevation
Int (auto_increment, primary key)	Varchar(250)	Varchar(250)	Double (because of decimal point, e.g. 25.2 km)	Double (because of decimal point, e.g. 800.9 metres)

So, for example, two routes would be populated as follows:

ID	Destination	Route_map	Distance	Elevation
1	Old Town	https://www.strava.com/routes/2927566549156874436	73.57 km	264 metres
2	Wicklow Town	https://www.strava.com/routes/3084910162487752404	95.39 km	773 metres

Future Work

Ideally, if I had more time, I would create a second table with competency level being my primary key.

Competency Level Group	Average Speed (Km/h)
Cyan	< 20
Green	>= 20 but < 23
Blue	>= 23 but < 26
Yellow	>= 26 but < 32
Red	>= 32

There will be a many-to-many relationship between the two tables, whereby each competency group could do each of the routes, and there are many routes for each competency group.

Using a bridging table, I could then construct a third table, comprising competency level group from one table, the destination from the second table, and I would estimate the duration of a cycle to that destination based on competency level.

Competency Level Group	Destination	Duration
Blue	Old Town	$= \text{Distance} / \text{Avg speed} \times (1 + \text{elevation}/2000)$ $= 73.57 / \text{AVG}(23,26) \times (1 + 264/2000)$ $= 3.40 \text{ hrs}$
Red	Old Town	$= 73.57 / (32) \times (1 + 264/2000)$ $= 2.60 \text{ hrs}$
Blue	Wicklow Town	$= 95.39 / \text{AVG}(23,26) \times (1 + 773/2000)$ $= 5.40 \text{ hrs}$
Red	Wicklow Town	$= 95.39 / (32) \times (1 + 773/2000)$ $= 4.13 \text{ hrs}$

In Python, I might write these tables something along the following lines:

```
Routes = {  
    1: {  
        "destination": "Old Town"  
        "route_map": "https://www.strava.com/routes/2927566549156874436"  
        "distance": 73.57  
        "elevation" : 264  
    },  
    2: {  
        "destination": "Wicklow Town"  
        "route_map": "https://www.strava.com/routes/3084910162487752404"  
        "distance": 95.39  
        "elevation" : 773  
    }  
}
```

```
Levels = {  
    "Cyan": {"Average Speed": 20},  
    "Green": {"Average Speed": 22},  
    "Blue": {"Average Speed": 25},  
    "Yellow": {"Average Speed": 28},  
    "Red": {"Average Speed": 32},  
}
```

The bridging table might take the form

```
 durations = {  
    "Blue": {"routes": [1,2], "duration"},  
    "Red": {"routes": [1,2], "duration"}  
}
```